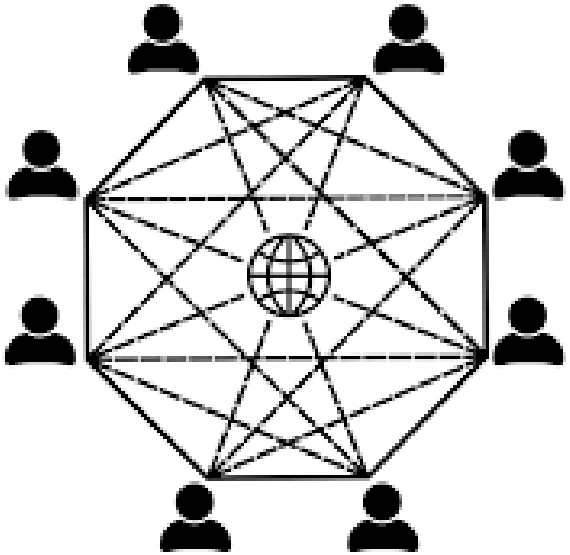
네트워크설계

2주차 - TCP/IP 기초와 네트워크 소켓



인공지능학과

그림참조: https://pxhere.com/ko/photo/1637436

▪ 네트워크 기초

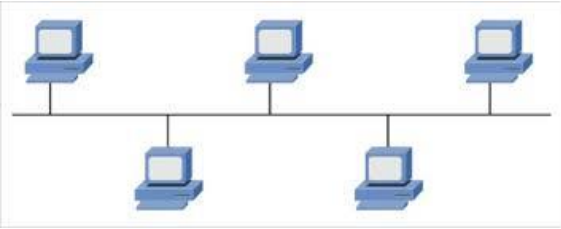
"소켓 프로그래밍을 하기 위해
기본적으로 알아야 하는 네트워크의 기초 사항들을 정리해 봅시다"

- 네트워크란 무엇인지?
- 어떤 구조로 되어 있는지?
- 각 계층은 어떤 역할을 하는지?
- 소켓 프로그래밍을 하기 위해 꼭 필요한 요소는 무엇인지?

■ 네트워크와 인터넷

❑ 컴퓨터 네트워크(Network)

- 분산되어 있는 컴퓨터들을 자원이나 정보를 공유하기 위하여 통신망으로 연결한 것
- 네트워킹: 네트워크 안에 연결된 노드들(컴퓨터들)간의 데이터 전송

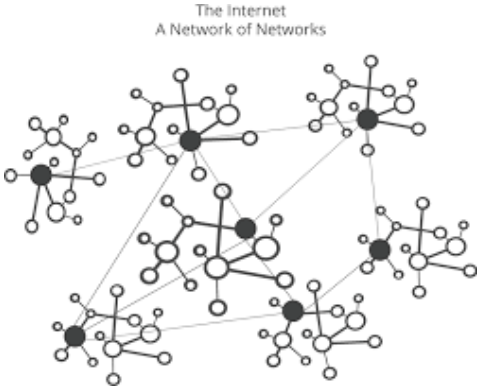


❑ 인터넷이란?

① "네트워크들의 네트워크 (Network of Networks)"
여러 통신망(Network)을 하나로 연결한다는 의미
→ 전 세계 컴퓨터들을 하나로 연결하는 거대한 컴퓨터 통신망을 의미
인터넷은 라우터 장비들의 집합

② 인터넷 프로토콜 → TCP/IP 프로토콜

③ 여러 컴퓨터가 각각 클라이언트와 서버로써 서로 연결



TCP/IP 4계층 구조

TCP/IP 4계층

① 응용 계층
② 전송 계층
③ 인터넷 계층
④ 링크 계층

① **응용 계층**
사용자 어플리케이션에서 사용하는 프로토콜
예) FTP, HTTP, SSH, Telnet, DNS, SMTP 등

② **전송계층**
- 통신 노드 간의 연결 제어 및 자료 송수신을 담당
- TCP(Transmission Control Protocol) : 연결형, 신뢰성
- UDP(User Datagram Protocol): 비연결형, 비신뢰성

③ **네트워크 계층**
- 최종 목적지까지 정확하게 연결되도록 경로 설정: 라우팅
- Host를 구분하기 위해 주소를 할당: IP 주소
- IP(Internet Protocol) : 호스트들과 네트워크에서 주소 관리, 패킷을 라우팅

④ **링크 계층**
- 물리적으로 데이터가 네트워크를 통해 어떻게 전송되는지를 정의, 물리주소(MAC주소)를 사용
  예) Ethernet, WiFi등

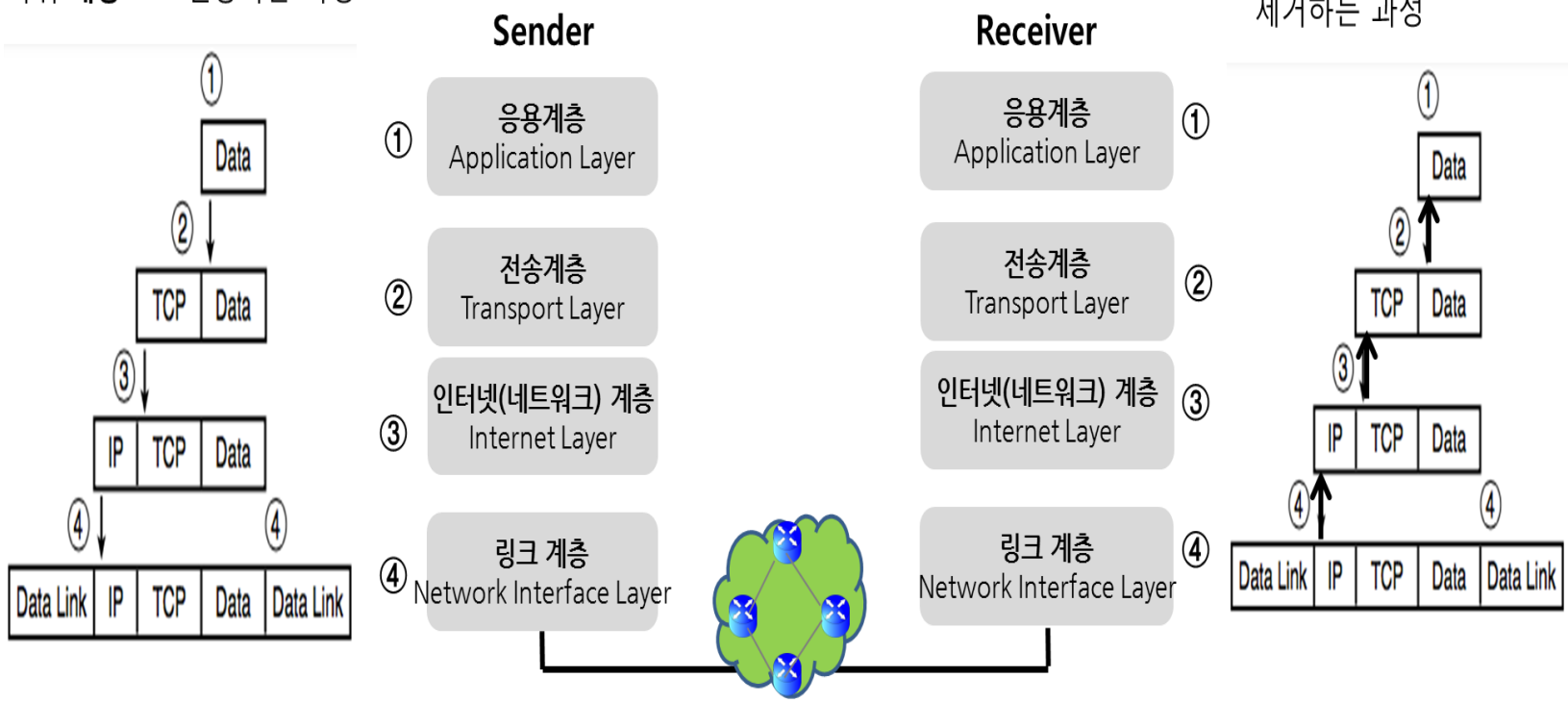
■ 프로토콜 계층 구조에서 데이터 전송과정

캡슐화 (Encapsulation)

컴퓨터 통신에서 상위 **계층**의 통신 프로토콜 정보(헤더)를 데이터에 추가하여 하위 **계층**으로 전송하는 과정

역캡슐화 (Decapsulation)

데이터를 받는 쪽에서는 계층마다 헤더를 하나씩 제거하는 과정



■ 프로토콜과 인터페이스

- 프로토콜: 서로 다른 호스트에 위치한 동일 계층끼리의 통신 규칙
- 인터페이스: 같은 호스트에 위치한 상하위 계층 사이의 규칙
- 서비스: 하위 계층이 상위 계층에 제공하는 인터페이스

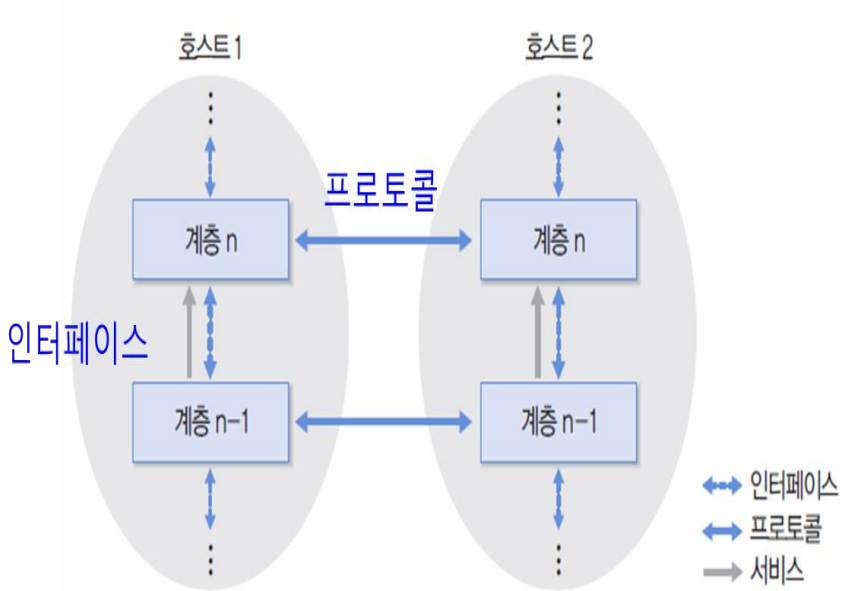
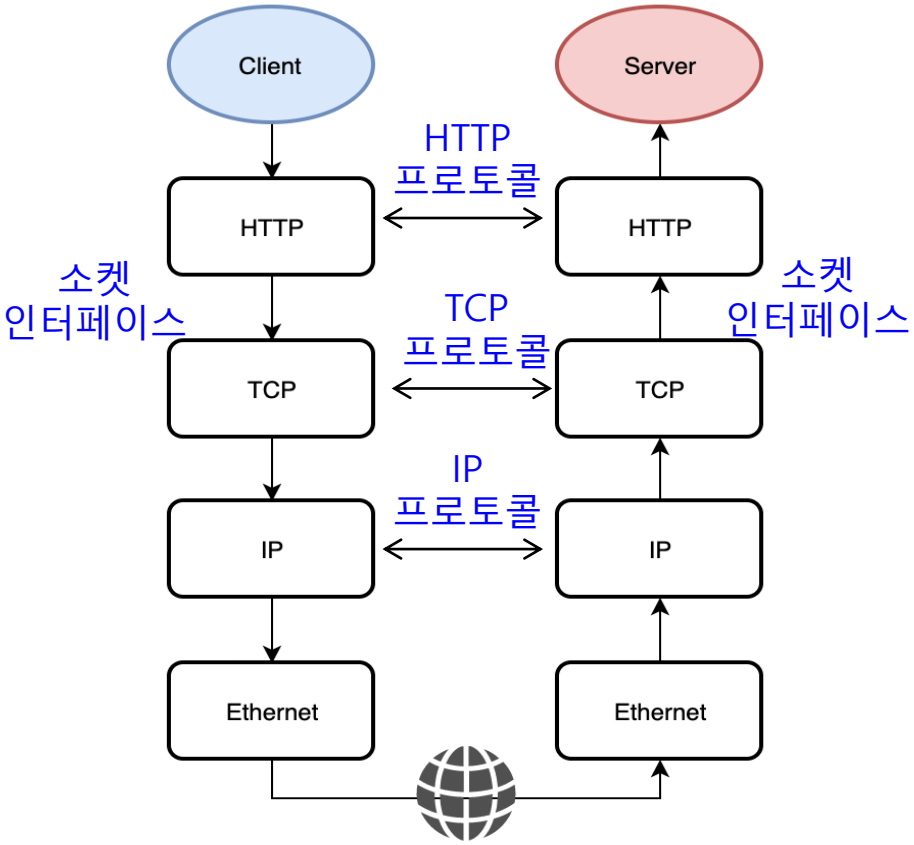


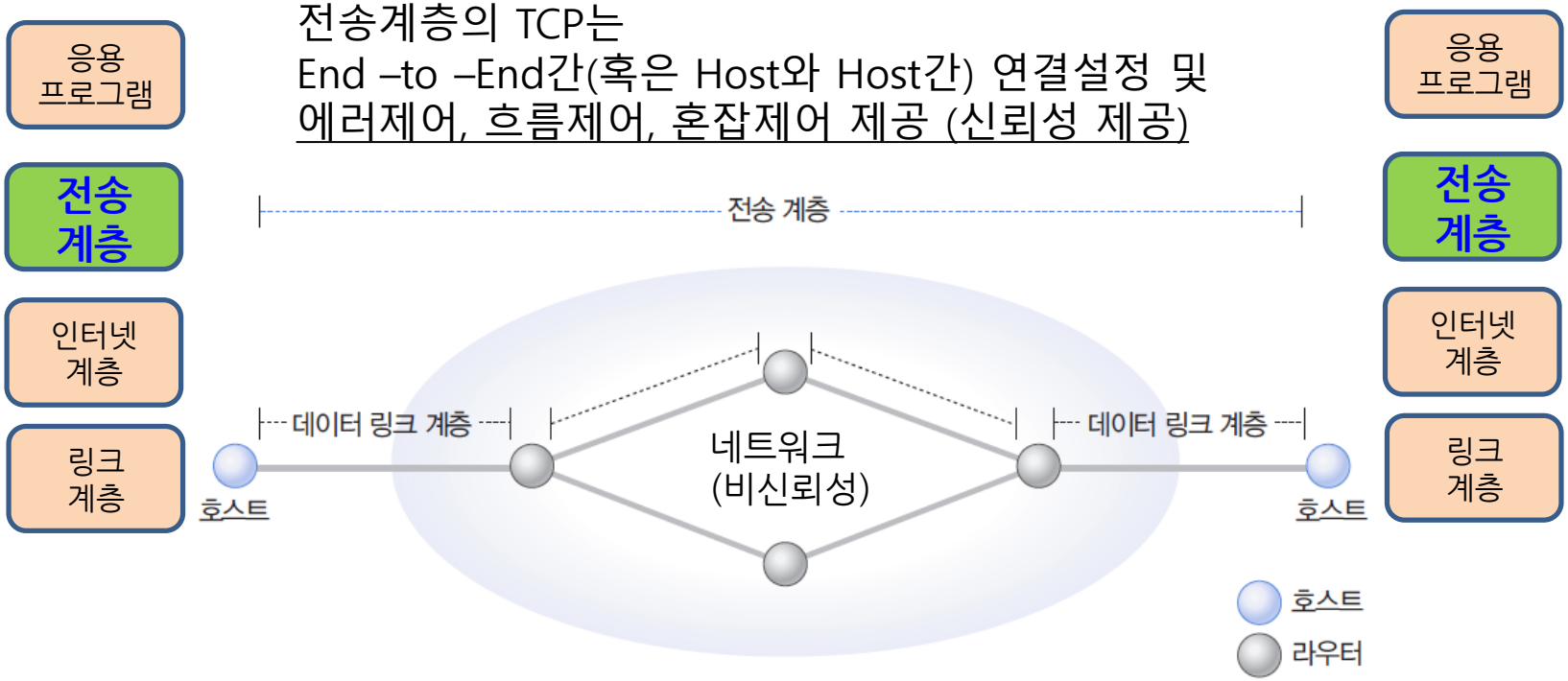
그림 1-5 인터페이스와 프로토콜



▪ 전송계층의 두 프로토콜 (TCP와 UDP)

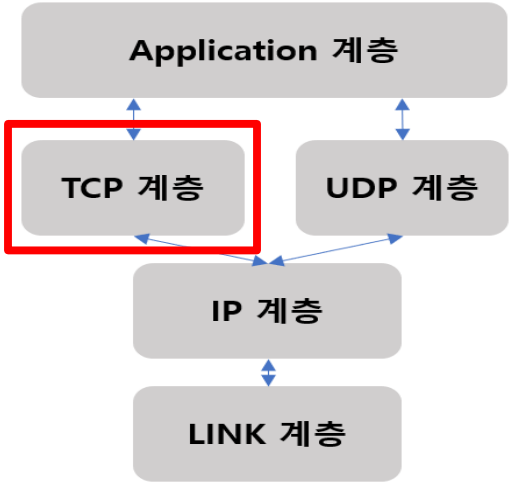
"전송계층은 비신뢰성 네트워크 위에서 신뢰성을 제공"

전송계층의 TCP는
End –to –End간(혹은 Host와 Host간) 연결설정 및
에러제어, 흐름제어, 혼잡제어 제공 (신뢰성 제공)



네트워크는 Best-effort 서비스 망이다
*자원이 허락하는 범위에서 최적의 통신 서비스 제공
단, 자원이 부족한 경우, 패킷손실, 패킷지연 발생
→ 네트워크는 비신뢰성

TCP(Transfer Control Protocol)



- **연결형**이며, **신뢰성**있는 데이터 통신을 가능하게 해주는 전송계층의 프로토콜

① **연결 지향적**(Connection-Oriented)

- 데이터를 전송하기 전에 먼저 수신 측과 연결을 설정

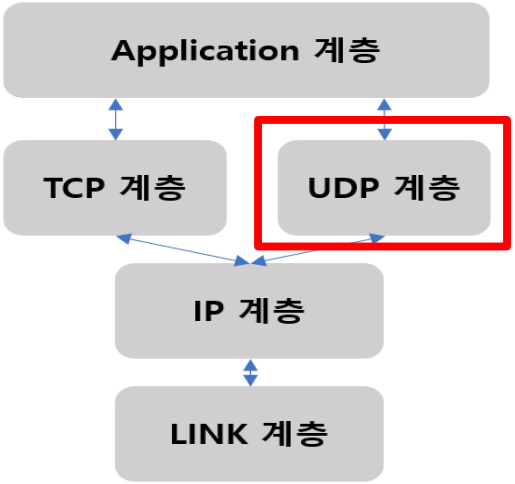
② **신뢰성 있는 데이터 통신**

- TCP는 송신 측과 수신 측 사이의 흐름제어 / 오류제어 / 혼잡제어 기능 제공

- 전송되는 데이터의 양을 조절하는 **흐름 제어**
- 데이터가 유실되거나 잘못된 데이터가 수신되었을 경우 대처하는 방법인 **오류 제어**
- 네트워크 혼잡에 대처하는 **혼잡 제어**

→ **응용계층은 전송계층의** TCP 덕분에 예외 상황에 크게 신경 쓰지 않고 응용계층 동작에 집중할 수 있음

UDP(User Datagram Protocol)



- **비연결형, 비신뢰성 서비스를 제공하는 전송계층 프로토콜**

- 흐름제어 기능이 없어 버퍼 오버플로Buffer Overflow에 의한 데이터 분실오류가 발생할 수 있음
- 에러제어 기능은 없으며 최소한의 오류검출 기능만 제공
  (패킷에러가 검출되면 폐기: 재전송하지 않음)

- **TCP보다 속도가 빠르다**
- 음성 및 비디오를 위한 실시간 스트리밍 서비스에 주로 사용 (그 외 DNS, DHCP등)

IP 주소 (IPv4 주소)

- 네트워크상에서 **컴퓨터(Host)를 식별하기 위해 부여된 주소**
- **32bit(4byte)** 길이로 구성된 논리적인 주소체계 (2^{32} 개 IP주소 배정가능)
- IP 주소는 **네트워크 주소**와 **호스트 주소** 두 부분으로 나뉜다. (IP주소 배정 규칙)

IP 주소 = Network ID + Host ID

	Network ID			Host ID
IPv4 주소	**192 .**	**168 .**	**10.**	**10**
서브넷 마스크	**255.**	**255.**	**255.**	**0**
	11111111	**11111111**	**11111111**	**00000000**

** 서브넷 마스크(AND 연산) 은 IP주소에서 네트워크 부분과 호스트 부분을 나누는 역할*

- Network(네트워크) ID : 네트워크 주소 (Prefix)
- Host(호스트) ID: 하나의 네트워크 내에 존재하는 호스트를 구분하기 위한 주소

1. 네트워크 기초 - IP 주소

- 내 컴퓨터의 IP 주소를 확인해 봅시다
  - 작업 표시줄에서 “검색” 아이콘 클릭 혹은 [Windows 키]를 눌러 검색창 열기
  - 검색창에 “cmd” 입력
  - 명령 프롬프트 창이 열리면, “ipconfig” 입력하고 [Enter]

```
C:\>
C:\>ipconfig

Windows IP 구성


이더넷 어댑터 이더넷:

   연결별 DNS 접미사. . . . :
   링크-로컬 IPv6 주소 . . . . : 7a91:ec8a::7a91:7a91:ec8a
   IPv4 주소 . . . . . . . . . . : 192.168.0.2
   서브넷 마스크 . . . . . . . . : 255.255.255.0
   기본 게이트웨이 . . . . . . . : 192.168.0.1
```

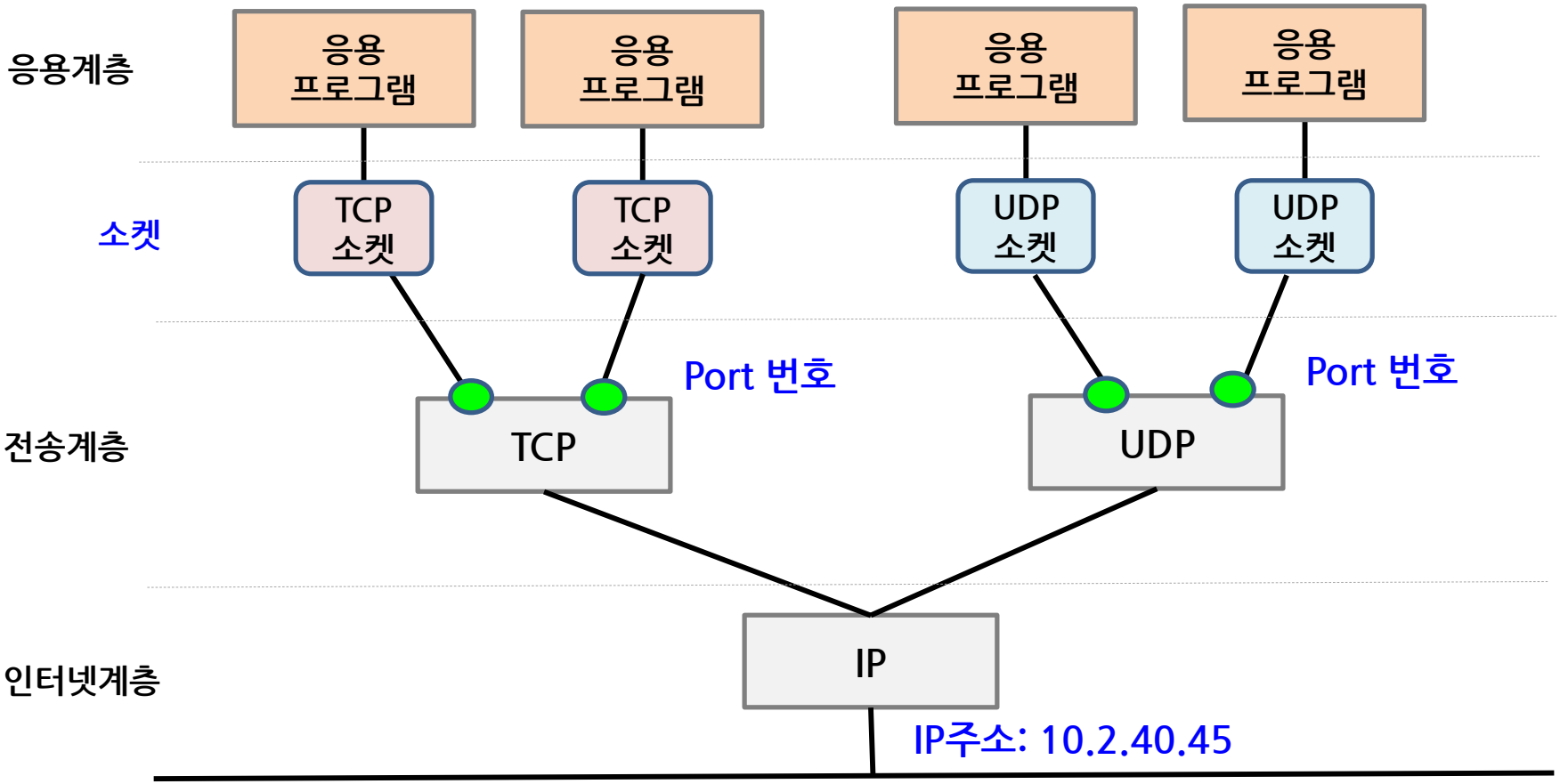
내 컴퓨터의 IP 주소

IP주소의 종류

종류	특징
공인 IP (Public IP)	• **전세계적으로 중복되지 않는 유일한 주소** • 인터넷 서비스 제공업체(ISP)가 사용자에게 할당하는 주소로 **인터넷 가능**
사설 IP (Private IP)	• **IP주소 부족문제를 해결하기 위해 등장** • 일반 가정이나 회사 등의 내부 네트워크에 할당된 IP 주소 • **사설 IP 주소만으로는 인터넷에 직접 연결할 수 없음** • 내부에서 외부로는 접근이 가능하지만 외부에서는 내부로 접근이 불가능 • IPv4 사설 IP 대역 - **10**.0.0.0 ~ **10**.255.255.255 (10.0.0.0/8) - **172.16.**0.0 ~ 172.31.255.255 (172.16.0.0/12) - **192.168**.0.0 ~ 192.168.255.255 (192.168.0.0/16)
루프백 IP (localhost)	• 네트워크상에서 자신을 나타내는 가상적인 주소 • **외부 네트워크를 사용하지 않고, 로컬에서 소프트웨어를 테스트하는 데 사용** • 대표적인 루프백 IP는 127.0.0.1 (localhost)를 사용
브로드캐스트 IP	• **네트워크에 있는 컴퓨터나 장비 모두에게 한 번에 데이터를 전송하는데 사용** • 255.255.255.255 또는 • IP 주소의 SUBNET 부분을 제외한 호스트 자리를 2진수 1로 채운 것
모든 IP	• 0.0.0.0**(모든 IP 주소)'는 내 컴퓨터에 연결된 모든 주소**

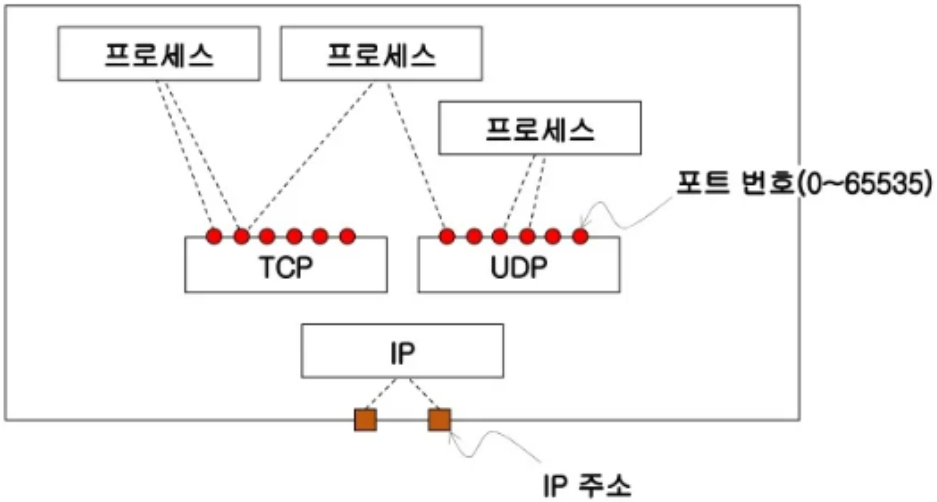
■ 네트워크 소켓

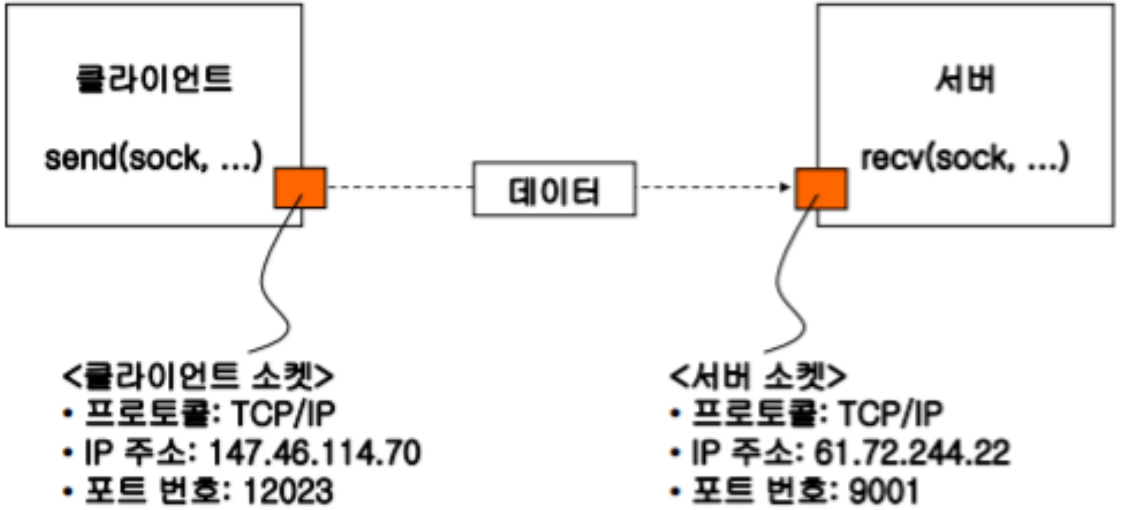
- 응용 프로그램간(서버와 클라이언트) 네트워크를 통해 서로 통신할 수 있도록 연결해 주는 소프트웨어(라이브러리)
- 응용계층과 전송계층 사이에 위치, 응용프로그램이 TCP/IP를 이용할 수 있도록 인터페이스 역할
- 소켓에는 IP 주소와 Port번호가 할당 (IP주소로 컴퓨터를 특정, Port번호로 프로세스를 특정)



소켓 구성요소 (전송계층 프로토콜, IP주소, 포트번호)

- 사용할 프로토콜 (TCP, UDP)
- 송신측 IP주소
- 송신측 포트번호
- 수신측 IP주소
- 수신측 포트번호





그림참조:https://www.slideshare.net/ssuser27542c/ch01-20694311

포트번호(Port number)

- IP주소를 통해 패킷 메시지가 목적지 컴퓨터까지 왔다면 이제 목적지 컴퓨터에서 어떤 프로그램(Application)에게 가야 하는지 알려주는 것이 포트번호 (Port number)
  - 포트번호: 0 ~ 65535

- Port 번호는 3가지로 구분

1) 널리 알려진 포트 (Well Known Port) : 0 ~ 1023

*어떤 특권을 가진 서비스에 사용되는 포트(시스템포트)
사용자가 직접 등록해서 사용할 수 없는 포트

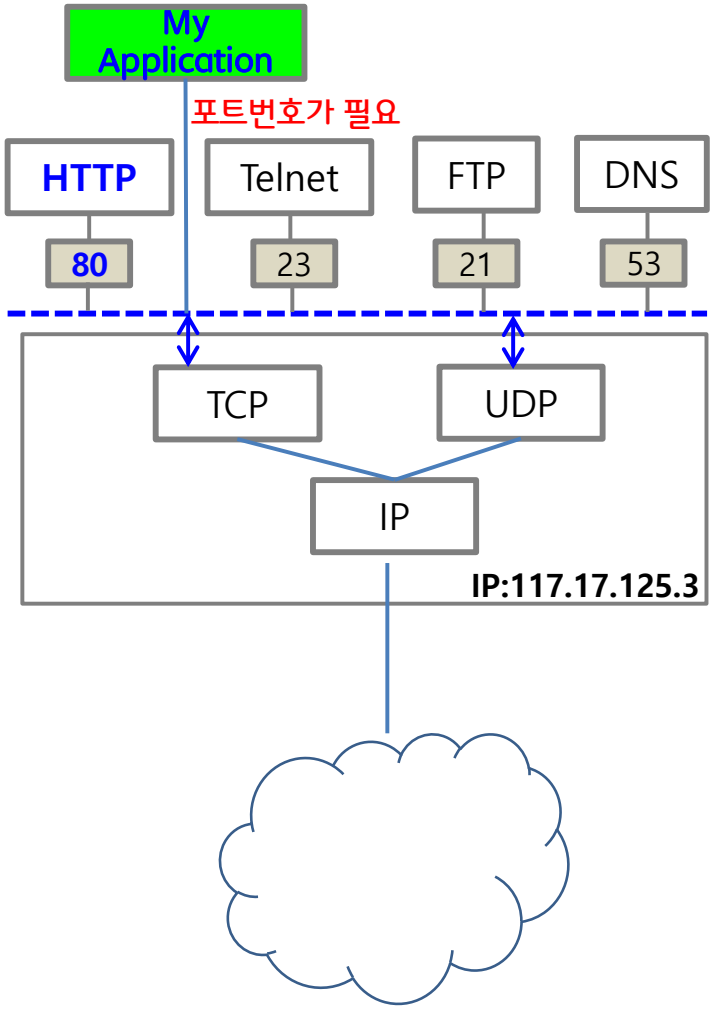
서비스	애플리케이션층 프로토콜	포트 번호	트랜스포트층 프로토콜
WWW	HTTP	80	TCP/UDP
WWW(보안 추가)	HTTPS	443	
전자메일(송신)	SMTP	25	
전자메일(수신)	POP3	110	
파일 전송	FTP	20 · 21	
원격 로그인	TELNET	23	
원격 로그인(보안)	SSH	22	
넷뉴스	NNTP	119	
네트워크 관리 DNS	DNS	53	
네트워크 관리 DHCP	DHCP	546 · 547	UDP
네트워크 관리 SNMP	SNMP	161 · 162	

2) 예약된 포트 (Registered port) : 1024 ~ 49151

*사용자가 직접 등록해서 사용할 수 있는 포트 (사용자포트)
클라이언트에서 사용할 수 있는 소켓 포트

3) 동적, 사설 포트 : 49152 ~ 65535

* 시스템에서 동적으로 할당하는 포트



■ 소켓의 타입 (통신방식: 전송계층 프로토콜)

```
SOCK_STREAM      /* 연결형 서비스, TCP */
SOCK_DGRAM       /* 비연결형 서비스, UDP */
SOCK_RAW
```

※ SOCK_RAW : IP 프로토콜을 직접 사용해 통신할 때 사용함

- 소켓 생성시 소켓의 타입을 지정 - 사용할 IP주소 체계와 통신방식 지정

서버 TCP 소켓 생성의 예

import socket

IP주소 형식 통신방식

server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)

- 첫 번째 인자(Address Family): 주소 체계를 지정하는 인자 (default는 IPv4)

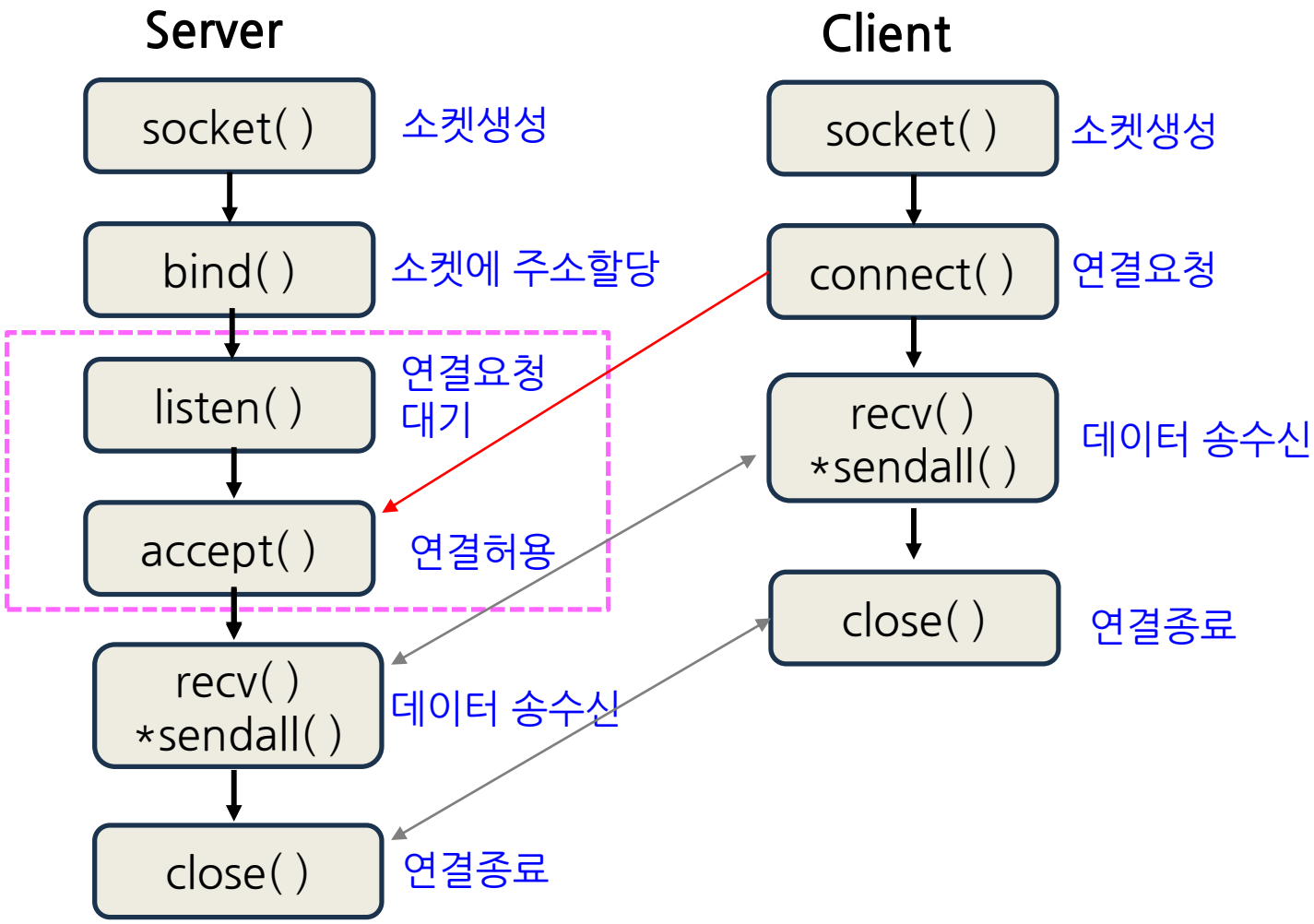
AF_INET = IP4v , AF_INET6 = IP6v

- 두 번째 인자(type): 소켓의 타입 (default는 TCP)

SOCK_STREAM = TCP , SOCK_DGRAM = UDP

■ 소켓 프로그래밍 절차

① 소켓생성 ②연결 설정 ③데이터 송수신 ④연결해제 순으로 수행



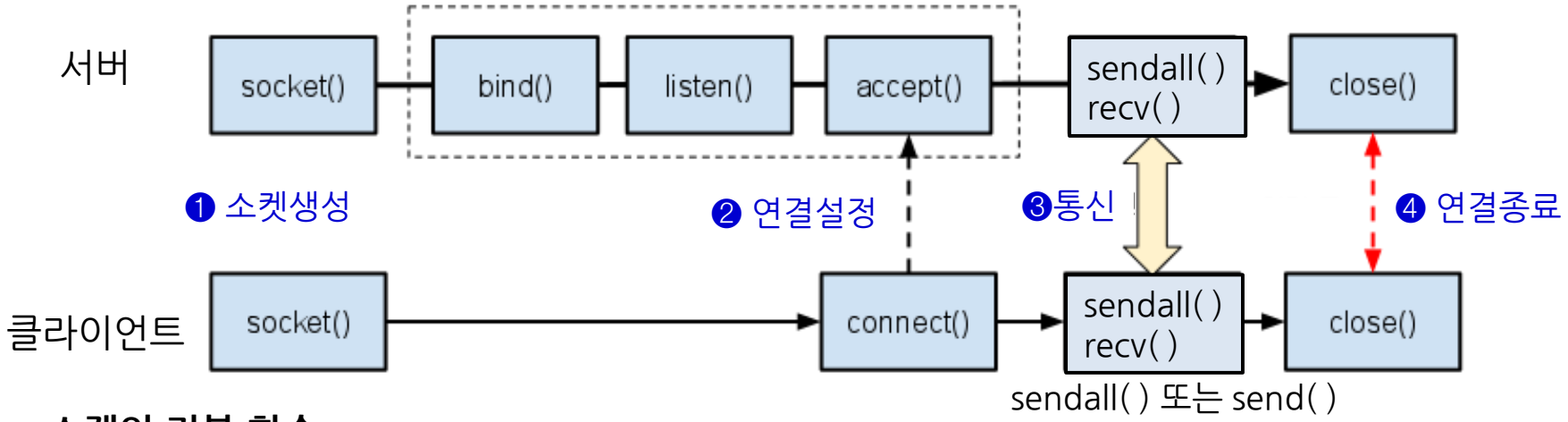
** 파이썬에서 소켓으로 메시지 보낼 때는 기존 send( ) 대신에 sendall( ) 사용 가능*
sendall( )은 파이썬 기반의 함수로 버퍼내 모든 데이터를 전송하는 기능제공

2. 네트워크 기초 - 소켓 개념과 사용법

- **TCP 소켓을 이용한 응용 프로그래밍 구성**

서버에서는
socket( ), bind( ), listen( ) 순으로
함수들을 호출하여 리스닝 소켓을 생성

클라이언트가 연결되면
accept( )에서 새로운 통신소켓을
생성하여 클라이언트와 통신시 사용



- **소켓의 기본 함수**

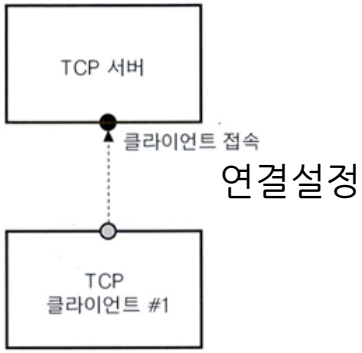
단계	서버	클라이언트
❶ 소켓생성	socket( ) : 소켓 생성 (리스닝 소켓)	socket( ) : 소켓 생성
❷ 연결설정	bind( ) : 소켓에 IP 주소와 포트번호 할당 listen( ) : 클라이언트 연결 대기 accept( ) : 연결수락 및 새로운 통신소켓 생성	connect( ) : 서버에 연결 요청
❸ 통신	sendall( )/recv( ) : 메시지 송수신 (통신 소켓)	sendall( )/recv( ) : 메시지 송수신
❹ 연결종료	close( ) : 소켓 닫기	close( ) : 소켓 닫기

TCP 소켓 프로그래밍 동작방식

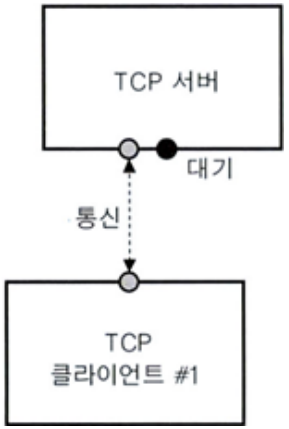
1) 서버 리스닝 소켓 생성 및 바인딩 (IP, PORT)

listen

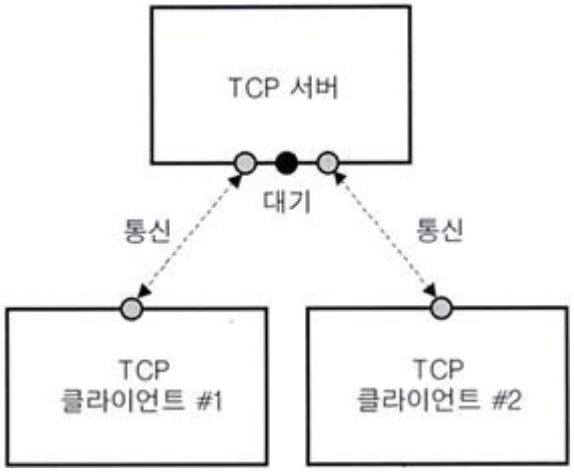
2) 클라이언트 소켓생성 및 접속



3) 서버는 통신을 위한 새로운 통신소켓 생성
* 이 소켓을 통해 데이터 송수신



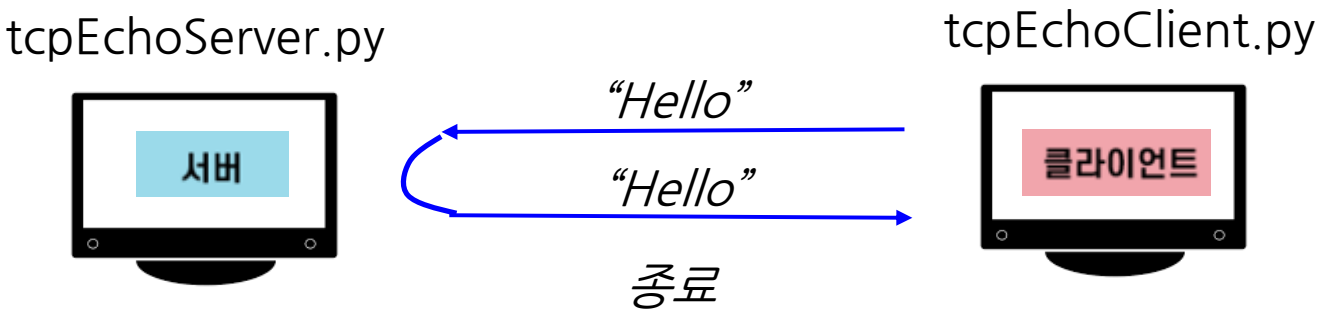
4) 서버는 다른 클라이언트 접속요청이 있으면
새로운 소켓 생성 (반복)



3. 소켓 프로그래밍 - TCP 에코 서버와 클라이언트

■ TCP 소켓을 이용한 에코(Echo) 서버와 클라이언트 구현하기

“ 에코 서버와 클라이언트란?
클라이언트가 메시지를 전송하면 서버는 그 메시지를 그대로 클라이언트로 전송하는 것 "

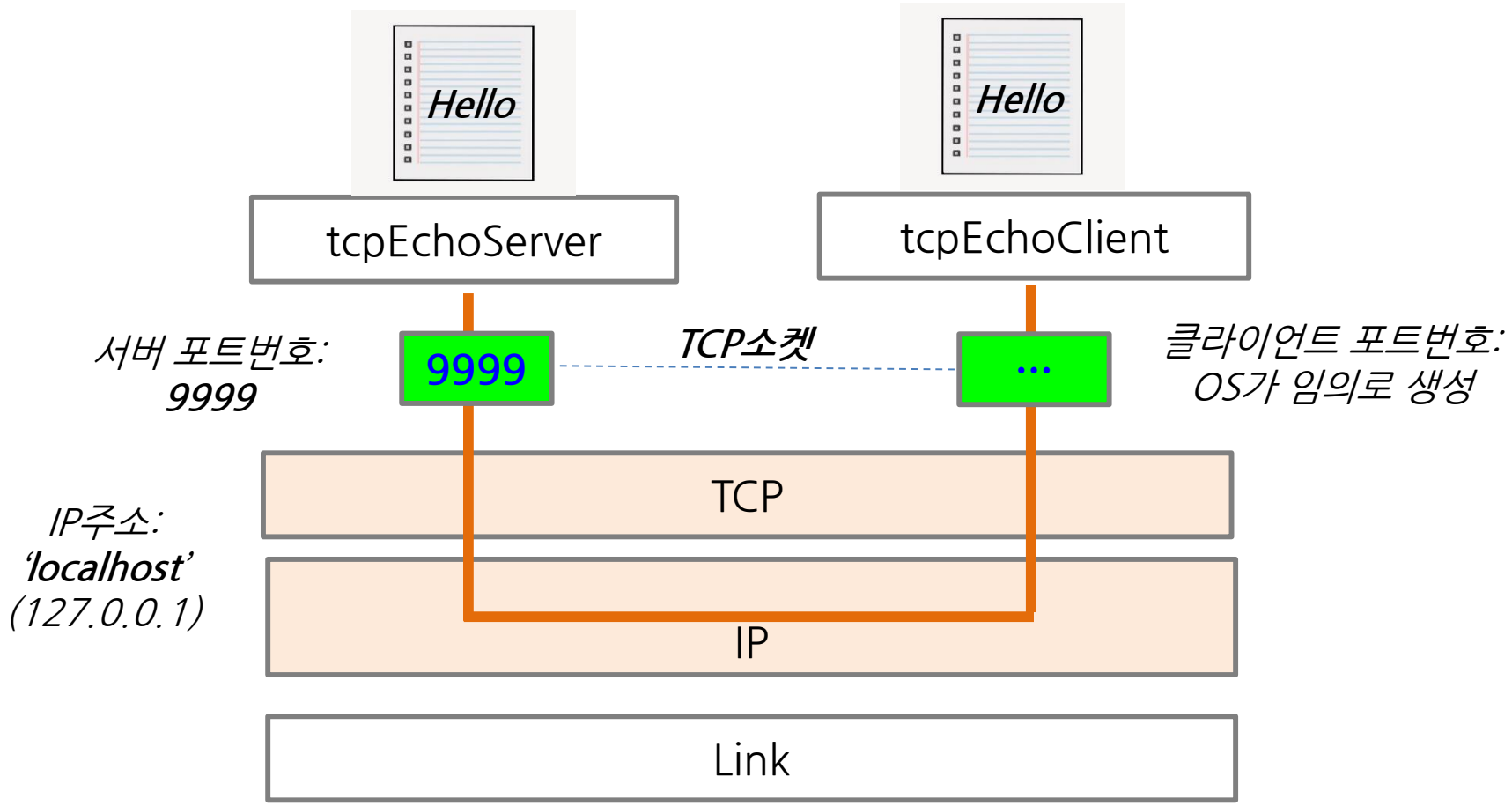



```
>> Echo Server start
>> Echo Server wait.............
>> ('127.0.0.1', 14536) is connected
>> rx msg: Hello
>> Echo Server End
```

```
>> Client start
>> Connect Server
>> send msg:  Hello
>> echo msg:  Hello
>> Client End
```

TCP 에코 서버 클라이언트 네트워킹 환경

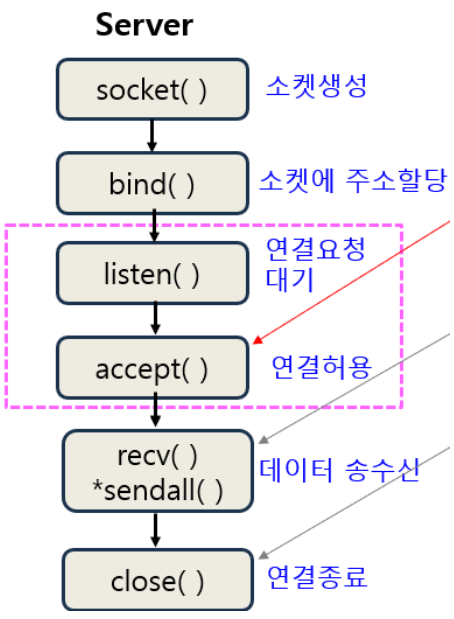
- IP주소는 'localhost' 사용:
  localhost는 loopback 주소로 내 컴퓨터 지칭하는 host 이름, 실제 IP주소는 127.0.0.1
  물리적인 링크를 사용하지 않고 IP단에서 바로 상위로 올려 보내 패킷을 외부로
  전송하지 않음



TCP 에코 서버 프로그램

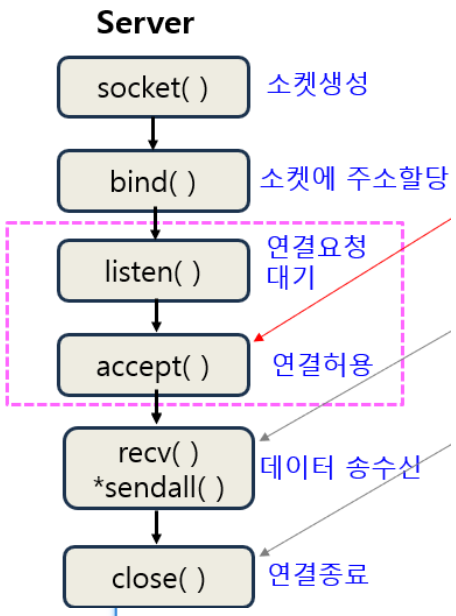
tcpEchoServer.py

```python
# TCP Echo server program

import socket

IP = 'localhost'  # loopback ip (127.0.0.1)
PORT = 9999
address = (IP, PORT)  # tuple로 주소정보 저장

print(">> Echo server start")

# socket() 서버 소켓 생성
# AF_INET:주소종류는 IPv4, SOCK_STREAM:통신타입은 TCP
server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)

# (optional code) winError 10048 에러 해결을 위한 추가 코드
server.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR,1)

# bind(주소정보) 소켓에 IP주소와 포트번호 지정
server.bind(address)
# listen() 클라이언트 접속요청 대기 상태
server.listen()
print(">> Echo server wait.........")

```



TCP 에코 서버 프로그램

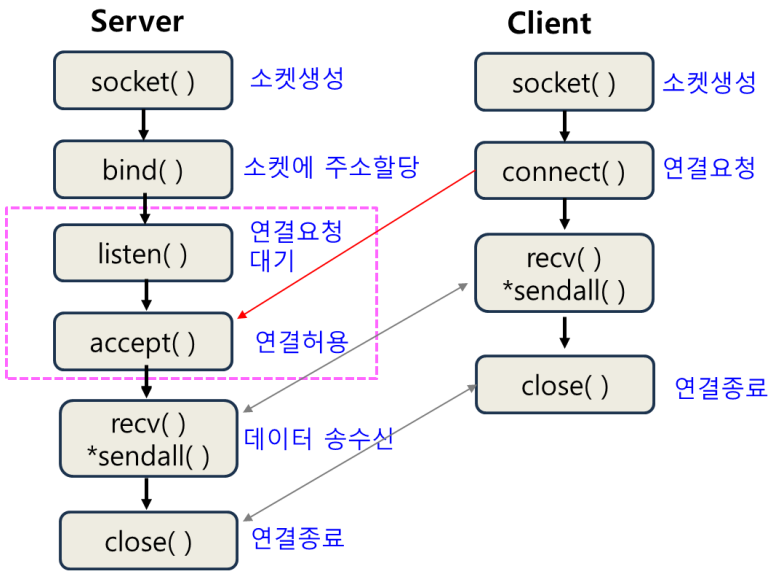
```
24 # accept() 클라이언트 접속 요청이 있을 시 수락
25 # 통신소켓과 클라이언트 주소정보를 반환
26 conn, addr = server.accept()
27 print(f">> {addr} is connected")
28
29 # recv(받을 메시지크기) 통신소켓에 데이터가 들어올때 까지 기다림
30 # 통신소켓에 데이터가 들어오면 메시지 반환
31 rxmsg = conn.recv(1024)
32 msg = rxmsg.decode('utf-8')  # 받을때는 디코딩
33 print(">> recv msg:", msg)
34
35 # sendall(보낼데이터) 통신소켓에 메시지 보내기
36 # 에코서버는 받은 데이터를 그대로 돌려줌
37 conn.sendall(msg.encode('utf-8'))  # 보낼때는 인코딩
38
39 # close() 서버 소켓 종료
40 conn.close()      # 통신소켓 종료
41 server.close()    # 리스닝소켓 종료
42
43 print(">> Echo server End")
```



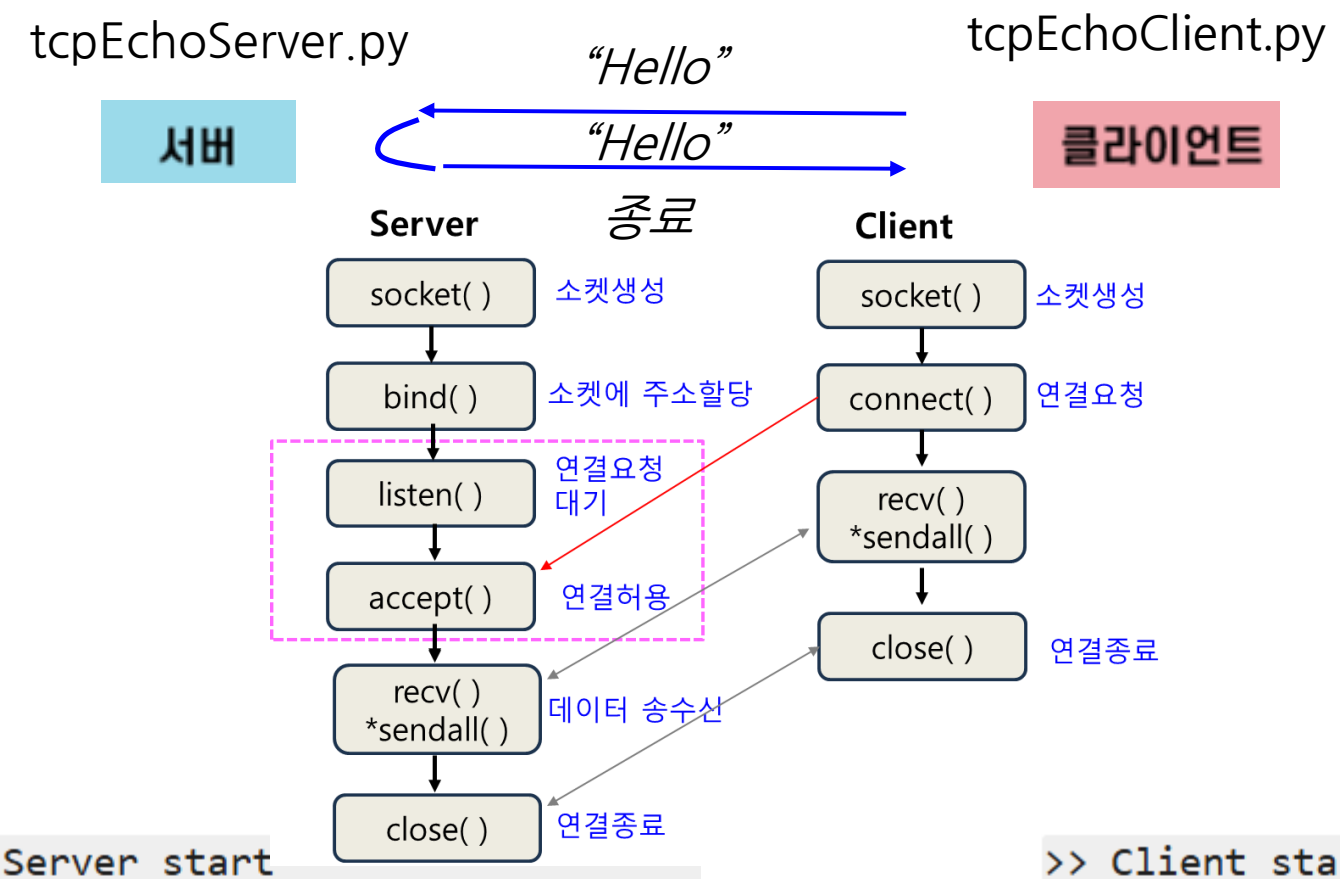
TCP 에코 클라이언트 프로그램

tcpEchoClient.py

```python
# TCP echo client program

import socket

IP = 'localhost'  # 서버 IP
PORT = 9999       # 서버 포트번호
address = (IP, PORT)  # tuple로 주소정보 저장

print(">> Echo client start")

# socket() 클라이언트 소켓 생성 (ipv4, tcp 소켓)
client = socket.socket(socket.AF_INET, socket.SOCK_STREAM)

# connect(서버주소정보) 서버소켓으로 연결요청
client.connect(address)
print(">> Connect server")

# sendall(보낼데이터) 서버로 메시지 보내기
msg = input(">> send msg: ")
client.sendall(msg.encode('utf-8')) # 보낼 때 인코딩

# recv(받을 메시지크기) 에코서버로부터 되돌아 온 메시지 수신
rxmsg = client.recv(1024)
msg = rxmsg.decode('utf-8')   # 받을때는 디코딩
print(">> recv msg: ", msg)

# close() 클라이언트 소켓 종료
client.close()
print(">> Client End")
```



TCP 에코 서버 클라이언트 프로그램 실행




```
>> Echo Server start
>> Echo Server wait.............
>> ('127.0.0.1', 14536) is connected
>> rx msg: Hello
>> Echo Server End
```

```
>> Client start
>> Connect Server
>> send msg:  Hello
>> echo msg:  Hello
>> Client End
```

4. 즐겨찾기 앱 제작하기

“ 앱 인벤터의 [웹뷰어] 컴포넌트를 이용해서
웹 사이트를 내 스마트폰 화면에 띄우는
‘즐겨찾기 앱’ 을 제작해 봅시다”

“버튼을 클릭하면 원하는 웹 사이트 바로가기 앱을 만들어 봅시다”

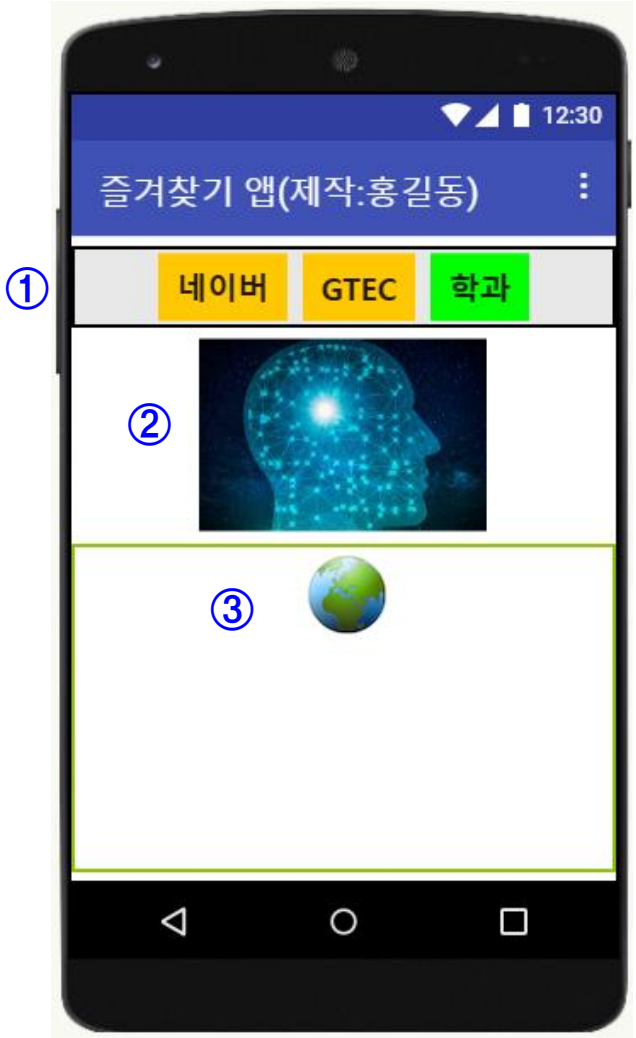
4. 즐겨찾기 앱 제작하기 - 프로젝트 시작하기

"시작은 새 프로젝트 만들기부터 "

❑ 프로젝트 이름: Websites_학번

4. 즐겨찾기 앱 제작하기 - 앱 디자인

1) 즐겨찾기 앱 디자인하기



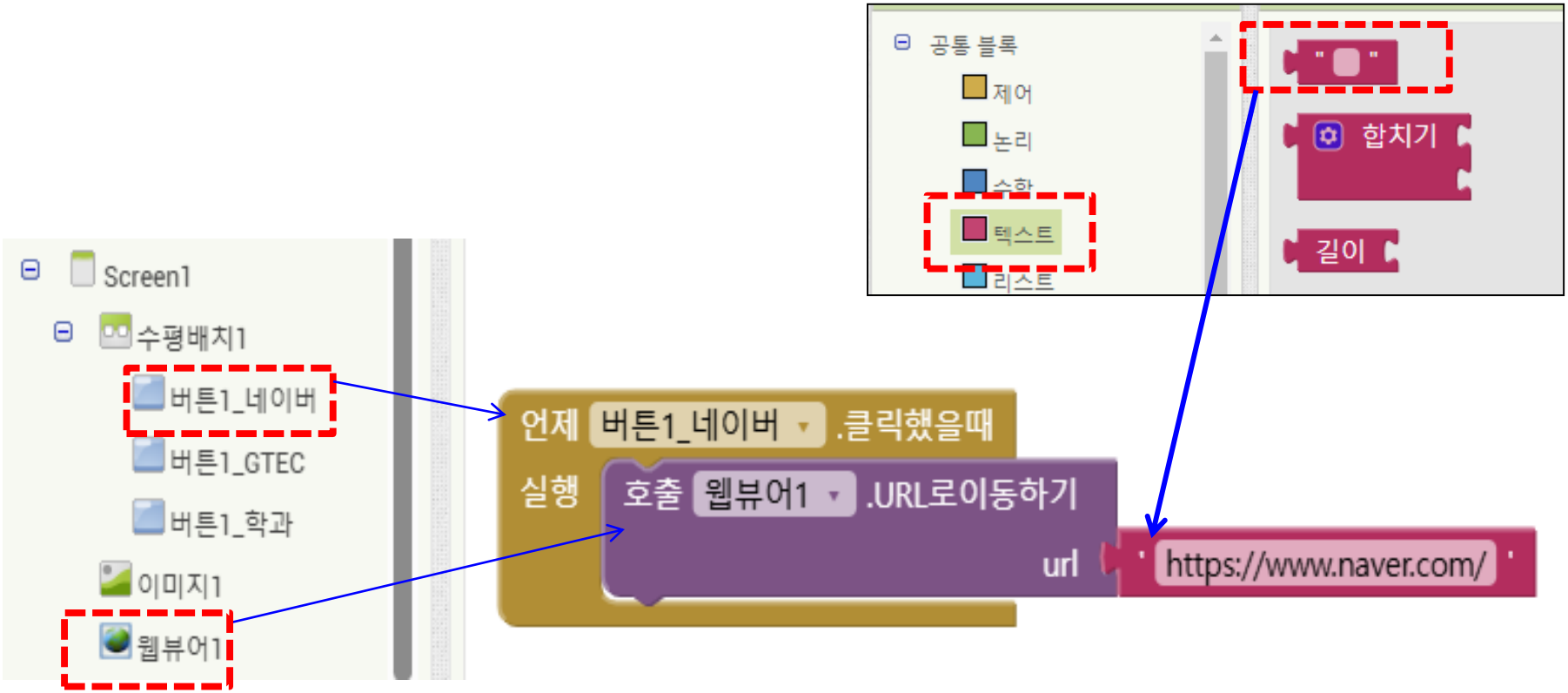
- 스크린 >> 속성 >> 수평정렬 - 가운데,
  제목- 즐겨찾기 (제작: 홍길동)

	컴포넌트	이름	속성
①	수평배치	수평배치1	수평, 수직정렬 - 가운데, 너비-부모요소
	버튼	버튼1_네이버	글꼴굵게, 글꼴크기 -20, 모양 - 직사각형 텍스트- 네이버, 텍스트정렬:가운데
	버튼	버튼1_ GTEC	글꼴굵게, 글꼴크기 -20, 모양 - 직사각형 텍스트- GTEC, 텍스트정렬:가운데
	버튼	버튼1_학과	글꼴굵게, 글꼴크기 -20, 모양 - 직사각형 텍스트- 학과, 텍스트정렬:가운데
②	이미지	이미지1	사진 - 이미지는 자유로 선택 높이-30퍼센트, 너비-자동
③	웹뷰어	웹뷰어1	홈URL - https://www.google.com/

4. 즐겨찾기 앱 제작하기 - 앱 코딩

2) 즐겨찾기 앱 코딩하기

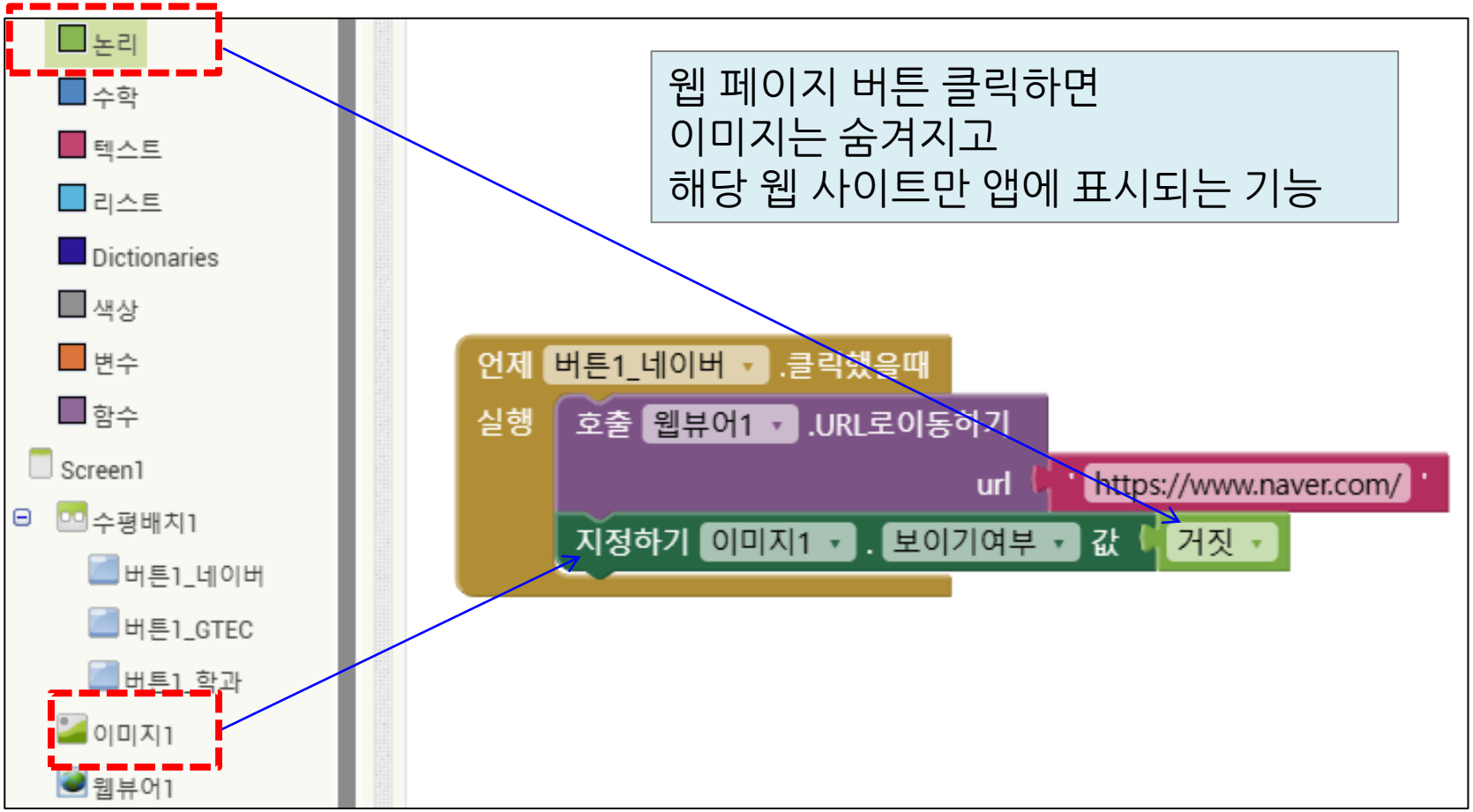
1. 버튼1_다음 클릭 > [언제 버튼1_네이버 클릭 했을때] 이벤트핸들러 블록 가져오기
2. 웹뷰어 클릭 > [호출 URL로 이동하기] 명령어 블록 연결하기
3. 텍스트 클릭 > [공백] 텍스트 가져와서 > https://www.naver.com/ 입력



4. 즐겨찾기 앱 제작하기 - 앱 디자인

2) 즐겨찾기 앱 코딩하기

1. 이미지 클릭 >> [지정하기(이미지1) 보이기여부] 블록 가져오기
2. 논리 클릭 >> 거짓 블록 가져오기



4. 즐겨찾기 앱 제작하기 - 앱 디자인

2) 즐겨찾기 앱 코딩하기

1. 다음과 동일한 방법으로 GTEC(대학) 추가 (URL: https://www.gtec.ac.kr/)
2. 같은 방법으로 인공지능학과 사이트 추가 (URL: https://www.gtec.ac.kr/ai/index.do)

4. 즐겨찾기 앱 제작하기 - 앱 실행

3) 즐겨찾기 앱 실행하기

- 버튼을 클릭하면 해당 웹 사이트로 이동하는지 확인

(참조) 앱 빌드하기

❑ 앱 빌드하기 (※안드로이드 폰만 가능합니다!)

❑ **프로젝트 빌드하기 : 앱 인벤터와 연결이 종료되더라도 폰에서 실행되도록 폰에 앱 설치**
→ 프로젝트가 완료 되었을 때 최종 작품을 스마트 폰에 설치하는 작업

- 빌드: 안드로이드 폰에 설치할 수 있는 앱을 만드는 과정
- 빌드의 결과물: *.APK (Android application Package)
  안드로이드 운영체제에서 설치와 배포를 위한 파일

- **앱 인벤터 2가지 빌드 방법**

① **QR코드 제공** : 'MIT AI2 Companion'앱에서 QR코드 스캔하여 내 폰에 설치 가능)

② **내 컴퓨터에 저장하기**: 설치파일을 다운받아 타인에게 배포 가능

2주차 과제제출 안내

[제작 예시]

❑ 2주차 과제 제출 안내

- 실습과제: 2주차에 실습한 즐겨찾기 앱에서

  1) 디자인 : 즐겨찾는 웹 사이트를 하나 추가해 보세요

  (※웹 사이트는 자유로 선택하세요)

  2) 코딩: 앱이 제대로 동작하도록 코드를 추가해 보세요

- 제출방법: 프로젝트 파일을 내 컴퓨터에 저장 후 CTL 2주차 과제 제출란에 제출

  (잘 안된 부분은 Comment 남겨 주세요!)

- 제출파일: Websites_학번.aia

  tcpEchoServer_이름.py

  tcpEchoClient_이름.py

Thank You